

ACCRA TECHNICAL UNIVERSITY

Department of Computer Science

BCS 404: Introduction to Data Science with Python

PROJECT WORK

Exploratory Data Analysis, Statistical Analysis and Machine Learning using a Real-World Dataset

(Titanic Dataset, Kaggle)

Student Name: ARTHUR SAMUEL HAGAN

Student ID: 01259850B

Lecturer: Dr. Joseph Dadzie

Academic Year: 2025/2026, Second Semester

Submission Deadline: Sunday, 19 July 2026

Table of Contents

<u>1. Introduction</u>	3
<u>2. Dataset Description</u>	3
<u>3. Methodology</u>	3
<u>3.1 Data Cleaning</u>	3
<u>3.2 Visualisation and Statistical Analysis</u>	3
<u>3.3 Machine Learning</u>	4
<u>4. Results</u>	4
<u>4.1 Visualisations</u>	4
<u>4.2 Statistical Analysis</u>	7
<u>4.3 Machine Learning Model Performance</u>	8
<u>5. Discussion</u>	8
<u>5.1 Major Findings</u>	8
<u>5.2 Statistical Insights</u>	9
<u>5.3 Machine Learning Results</u>	9
<u>5.4 Limitations of the Study</u>	9
<u>5.5 Recommendations</u>	9
<u>6. Conclusion</u>	9
<u>7. References</u>	10
<u>8. Appendix: Python Code</u>	11

1. Introduction

This report presents a complete data science workflow applied to the Titanic dataset, covering data acquisition, cleaning, exploratory visualisation, statistical analysis, and machine learning classification. The purpose of the project is to demonstrate practical competence in using Python libraries, including Pandas, NumPy, Matplotlib, Seaborn, and to analyse a real-world dataset and draw meaningful conclusions from it.

The Titanic dataset is a well-known benchmark dataset in the data science community, containing demographic and travel details of passengers aboard the RMS Titanic, along with whether each passenger survived the sinking. This project aims to identify the key factors associated with survival and to build a predictive model capable of estimating a passenger's likelihood of survival based on those factors.

2. Dataset Description

The dataset was obtained from the Kaggle "Titanic - Machine Learning from Disaster" competition (<https://www.kaggle.com/competitions/titanic/data>). The training set (train.csv) used in this project contains 891 passenger records and 12 variables, described below.

Variable	Description
PassengerId	Unique identifier for each passenger
Survived	Survival outcome (0 = No, 1 = Yes); this is the target variable
Pclass	Ticket class (1 = 1st, 2 = 2nd, 3 = 3rd), used as a proxy for socio-economic status
Name	Passenger's full name
Sex	Passenger's sex
Age	Age in years
SibSp	Number of siblings/spouses aboard
Parch	Number of parents/children aboard
Ticket	Ticket number
Fare	Passenger fare
Cabin	Cabin number (mostly missing)
Embarked	Port of embarkation (C = Cherbourg, Q = Queenstown, S = Southampton)

3. Methodology

3.1 Data Cleaning

Missing values were identified in three columns: Cabin (77.1%), Age (19.9%), and Embarked (0.2%). Age was imputed using the median age within each passenger class, since age correlates with class. Embarked was imputed using its mode, given the negligible number of missing entries. Cabin was too sparse to impute meaningfully, so it was converted into a binary feature, Has_Cabin, indicating whether cabin information was recorded, and the original column was dropped. No duplicate rows were found in the dataset.

3.2 Visualisation and Statistical Analysis

Six visualisations were produced to explore the distribution and relationships between key variables: a histogram of ages, a bar chart of passenger class distribution, a boxplot of age by class, a scatter plot of age versus fare, a correlation heatmap, and a pairplot of key numerical variables. Descriptive statistics, frequency distributions, and a full correlation analysis were then computed to quantify these relationships.

3.3 Machine Learning

A Logistic Regression classifier was trained to predict passenger survival. Categorical variables (Sex, Embarked) were label-encoded, and eight predictor variables were selected: Pclass, Sex, Age, SibSp, Parch, Fare, Embarked, and Has_Cabin. The dataset was split into an 80% training set (712 passengers) and a 20% testing set (179 passengers) using stratified sampling to preserve the survival class balance. The model was trained on the training set and evaluated on the held-out test set using accuracy, a confusion matrix, and a classification report.

4. Results

4.1 Visualisations

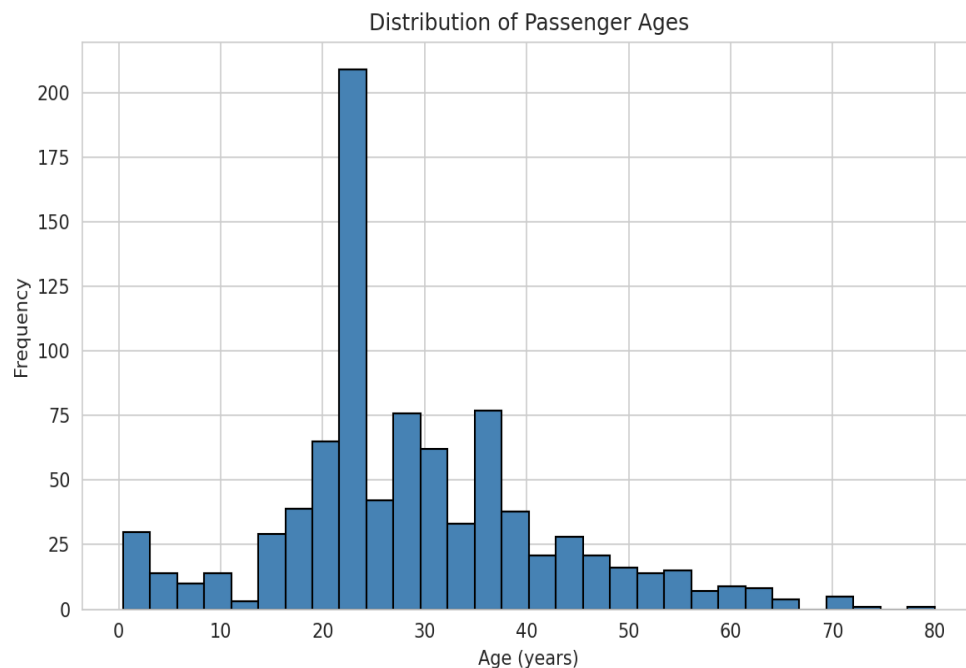


Figure 1: Distribution of passenger ages.

The age distribution is right-skewed, with most passengers aged between 20 and 40, and a smaller cluster of infants and young children.

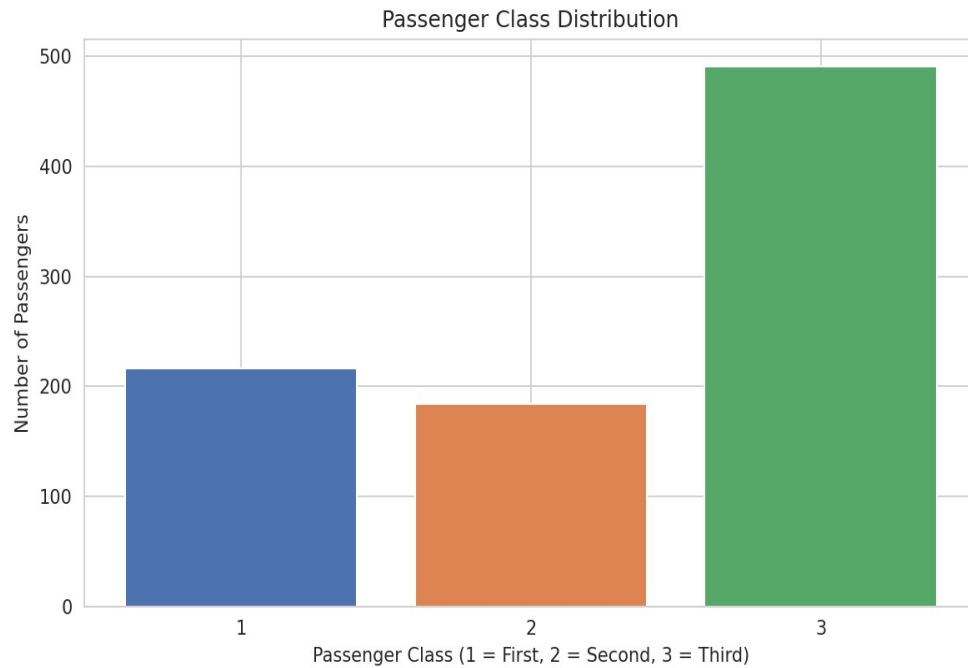


Figure 2: Passenger class distribution.

Third-class passengers formed the largest group (55.1%), followed by first class (24.2%) and second class (20.7%).

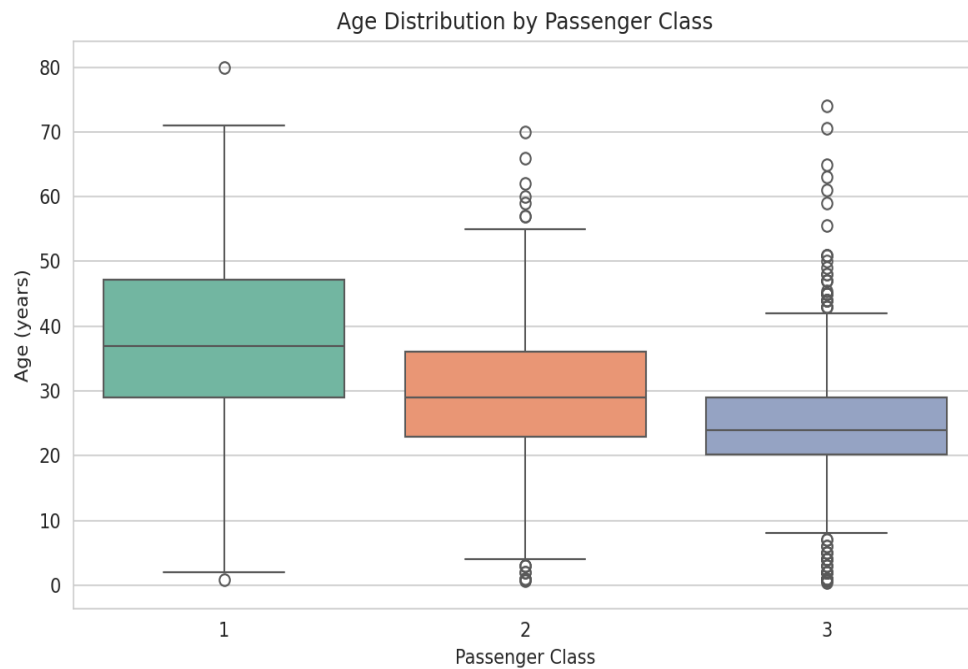


Figure 3: Age distribution by passenger class.

Median age decreased with passenger class: first-class passengers were oldest on average, third-class passengers were youngest.

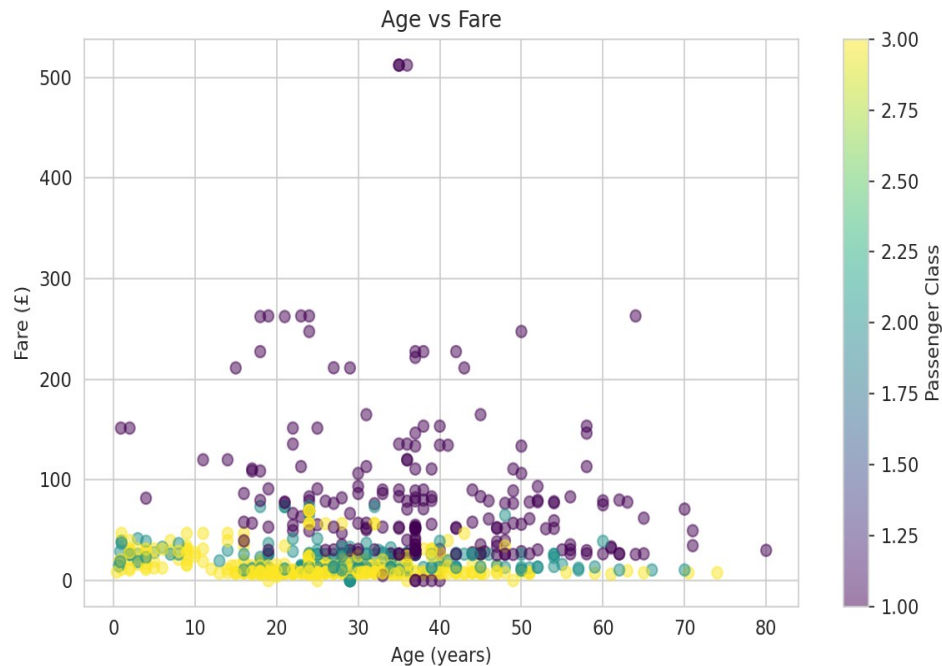


Figure 4: Age versus Fare, coloured by passenger class.

No strong linear relationship exists between age and fare, though high-fare outliers are concentrated among first-class passengers.

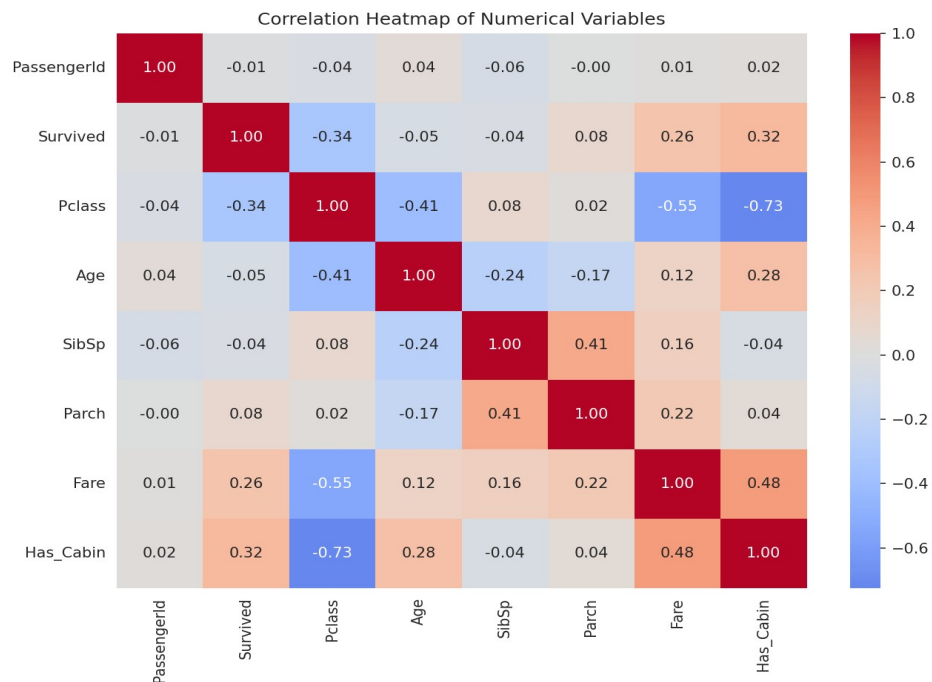


Figure 5: Correlation heatmap of numerical variables.

The strongest positive correlation was between Fare and Has_Cabin ($r = 0.48$). The strongest negative correlation was between Pclass and Has_Cabin ($r = -0.73$). Pclass also correlated negatively with Survived ($r = -0.34$).

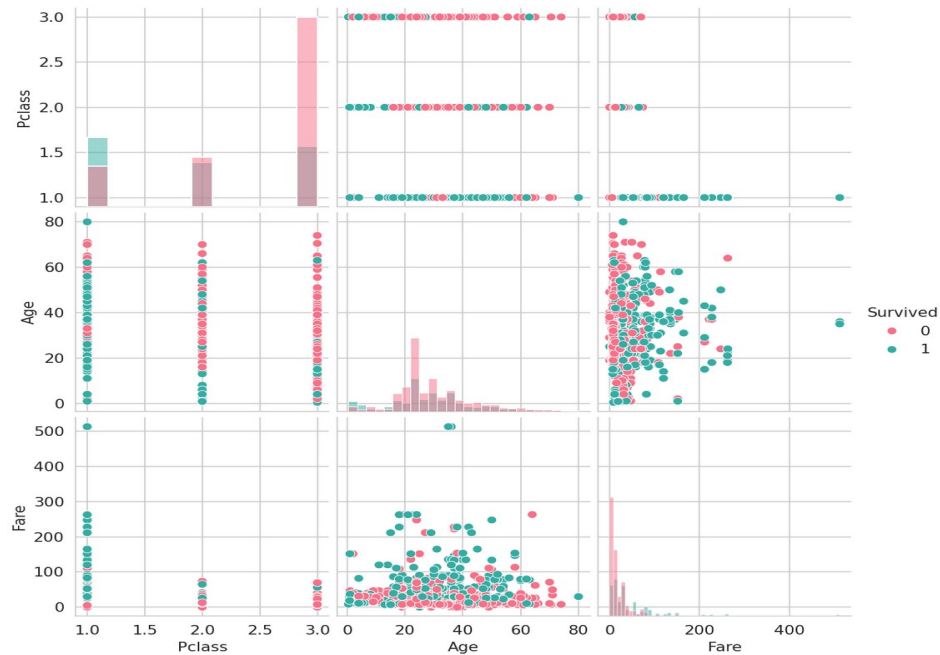


Figure 6: Pairplot of Survived, Pclass, Age, and Fare.

Survivors were more concentrated among lower Pclass values (first class) and higher fares, while age alone did not clearly separate survivors from non-survivors.

4.2 Statistical Analysis

Descriptive statistics showed a mean survival rate of 38.4% (549 passengers did not survive, 342 survived). The sample was 64.8% male and 35.2% female. Frequency distributions and correlation analysis confirmed the relationships described above, with Pclass showing the strongest correlation with the survival outcome among all variables directly linked to Survived.

4.3 Machine Learning Model Performance

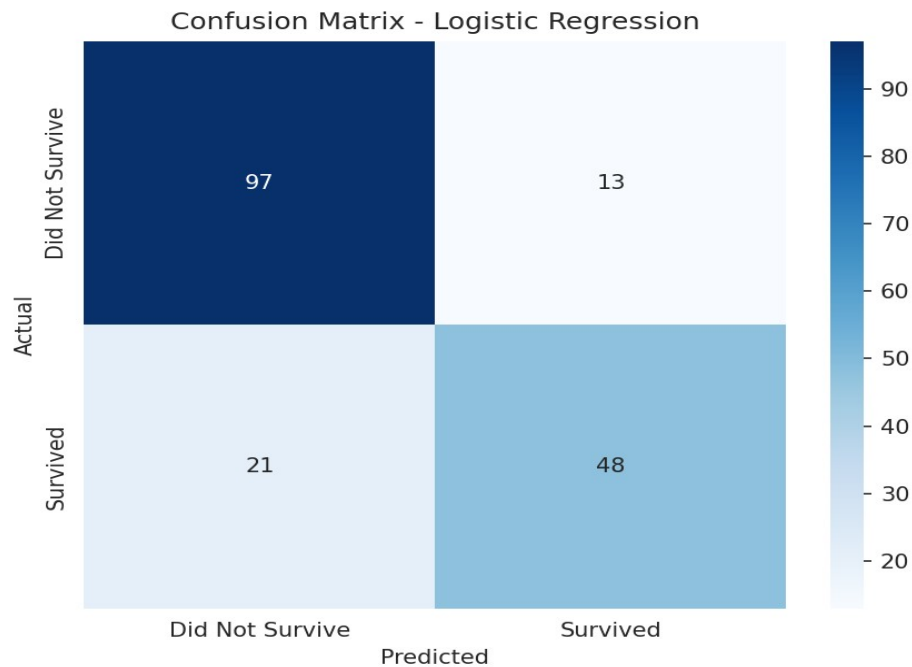


Figure 7: Confusion matrix for the Logistic Regression model.

The Logistic Regression model achieved an accuracy of 81.01% on the test set. The confusion matrix shows 97 true negatives, 13 false positives, 21 false negatives, and 48 true positives. Precision, recall, and F1-score for each class are summarised below.

Class	Precision	Recall	F1-score	Support
Did Not Survive	0.82	0.88	0.85	110
Survived	0.79	0.70	0.74	69
Accuracy			0.81	179

Feature coefficients showed that Sex (-2.59) and Pclass (-0.87) were the strongest predictors of survival, with Has_Cabin (+0.83) also having a notable positive effect. This indicates that being male and travelling in a lower class substantially reduced the odds of survival, while having recorded cabin information (associated with higher-class travel) increased it.

5. Discussion

5.1 Major Findings

Survival on the Titanic was strongly associated with passenger sex and class. Female passengers and first-class passengers had markedly higher survival rates than male and third-class passengers. This is reflected in the Logistic Regression model, where Sex and Pclass were the two strongest predictors of survival, with being male and travelling in a lower class both sharply reducing the odds of survival.

5.2 Statistical Insights

- The strongest positive correlation in the dataset was between Fare and Has_Cabin ($r = 0.48$), showing that passengers who paid more were far more likely to have a recorded cabin.
- The strongest negative correlation was between Pclass and Has_Cabin ($r = -0.73$), confirming that cabin allocation was almost entirely a first/second-class privilege.
- Pclass correlated negatively with Survived ($r = -0.34$), the strongest relationship any variable had with the survival outcome, reinforcing that class-based access (e.g., to lifeboats) played a major role in who survived.

5.3 Machine Learning Results

The Logistic Regression classifier achieved an accuracy of 81.01% on the held-out test set. The model correctly identified 97 of 110 non-survivors (88% recall) and 48 of 69 survivors (70% recall), meaning the model performs somewhat better at predicting non-survival than survival, likely because non-survivors were more common in the training data. Precision and recall were reasonably balanced overall (weighted F1-score of 0.81), indicating a generally reliable model despite its simplicity.

5.4 Limitations of the Study

- Missing Age values were imputed using the median age per passenger class rather than true observed values, introducing some estimation uncertainty.
- The Cabin variable was too sparse (77% missing) to use directly, so it was reduced to a binary Has_Cabin flag, which may lose finer-grained information.
- Logistic Regression assumes a linear relationship between predictors and the log-odds of survival, which may not capture more complex, non-linear interactions (e.g., between age and class).
- The dataset is specific to the Titanic disaster, so findings should not be generalised to other maritime accidents or contexts without caution.

5.5 Recommendations

- Future work could compare Logistic Regression against more flexible models, such as Random Forest or Gradient Boosting, to test whether predictive accuracy improves.
- Additional feature engineering, for example extracting passenger titles from the Name column or combining SibSp and Parch into a single family-size feature, could improve model performance.
- More advanced imputation methods (e.g., regression-based imputation) could be explored for Age instead of simple group-median filling.

6. Conclusion

This project applied a complete data science workflow to the Titanic dataset, from data acquisition and cleaning through to exploratory visualisation, statistical analysis, and predictive modelling. The analysis confirmed that passenger sex and class were the dominant factors influencing survival, consistent with historical accounts of the disaster. A Logistic Regression model built on eight predictor variables achieved 81% accuracy in predicting survival on unseen data, demonstrating that even a simple, interpretable model can capture the key patterns in this dataset. While the study has some limitations, particularly around missing data imputation and the linear assumptions of Logistic Regression, it successfully meets the project's objective of demonstrating practical competence in Python-based data science techniques on a real-world dataset.

7. References

Kaggle. (n.d.). Titanic - Machine Learning from Disaster. Retrieved from <https://www.kaggle.com/competitions/titanic/data>

McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90-95.

Waskom, M. (2021). Seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021.

8. Appendix: Python Code

The full Python code used for this analysis, extracted from the accompanying Jupyter Notebook (Titanic_Data_Science_Project.ipynb), is provided below.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (8, 5)

# Load dataset
df = pd.read_csv("data/train.csv")
print("Dataset loaded successfully.")

# ---

# Dataset dimensions
print("Dataset dimensions (rows, columns):", df.shape)

# ---

# Column names
print("Column names:")
print(df.columns.tolist())

# ---

# First five observations
df.head()

# ---

# Data types
df.dtypes

# ---

# Detect missing values
missing = df.isnull().sum()
missing_percent = (missing / len(df)) * 100
missing_summary = pd.DataFrame({'Missing Count': missing, 'Missing %':
missing_percent.round(2)})
missing_summary[missing_summary['Missing Count'] > 0].sort_values('Missing Count',
ascending=False)

# ---

# Handle missing values

# 1. Age: impute with the median age per passenger class (Pclass), since age tends to
# correlate with class (wealthier/older passengers often traveled in higher
classes)
df['Age'] = df.groupby('Pclass')['Age'].transform(lambda x: x.fillna(x.median()))

# 2. Embarked: only 2 missing values -> impute with the mode (most frequent port)
```

```

df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])

# 3. Cabin: over 77% missing, too sparse to impute meaningfully.
# Instead of dropping the column entirely, create a binary indicator of whether
# cabin information was recorded, then drop the raw Cabin column.
df['Has_Cabin'] = df['Cabin'].notnull().astype(int)
df.drop(columns=['Cabin'], inplace=True)

print("Missing values after cleaning:")
print(df.isnull().sum())

# ---

# Detect duplicated observations
duplicate_count = df.duplicated().sum()
print(f"Number of duplicated rows: {duplicate_count}")

# ---

# Remove duplicates if any exist
if duplicate_count > 0:
    df.drop_duplicates(inplace=True)
    print(f"Duplicates removed. New shape: {df.shape}")
else:
    print("No duplicate rows were found; no action needed.")

# ---

# 1. Histogram of passenger ages
plt.figure(figsize=(8,5))
plt.hist(df['Age'], bins=30, color='steelblue', edgecolor='black')
plt.title('Distribution of Passenger Ages')
plt.xlabel('Age (years)')
plt.ylabel('Frequency')
plt.tight_layout()
plt.savefig('outputs/fig1_age_histogram.png', dpi=150)
plt.show()

# ---

# 2. Bar chart showing passenger class distribution
plt.figure(figsize=(8,5))
class_counts = df['Pclass'].value_counts().sort_index()
plt.bar(class_counts.index.astype(str), class_counts.values,
color=['#4C72B0', '#DD8452', '#55A868'])
plt.title('Passenger Class Distribution')
plt.xlabel('Passenger Class (1 = First, 2 = Second, 3 = Third)')
plt.ylabel('Number of Passengers')
plt.tight_layout()
plt.savefig('outputs/fig2_class_bar.png', dpi=150)
plt.show()

# ---

# 3. Boxplot of age by passenger class
plt.figure(figsize=(8,5))
sns.boxplot(x='Pclass', y='Age', data=df, palette='Set2')
plt.title('Age Distribution by Passenger Class')
plt.xlabel('Passenger Class')
plt.ylabel('Age (years)')
plt.tight_layout()
plt.savefig('outputs/fig3_age_by_class_box.png', dpi=150)
plt.show()

```

```

# ---

# 4. Scatter plot of Age vs Fare
plt.figure(figsize=(8,5))
plt.scatter(df['Age'], df['Fare'], alpha=0.5, c=df['Pclass'], cmap='viridis')
plt.title('Age vs Fare')
plt.xlabel('Age (years)')
plt.ylabel('Fare (£)')
plt.colorbar(label='Passenger Class')
plt.tight_layout()
plt.savefig('outputs/fig4_age_vs_fare_scatter.png', dpi=150)
plt.show()

# ---

# 5. Correlation heatmap
plt.figure(figsize=(9,7))
numeric_df = df.select_dtypes(include=[np.number])
corr = numeric_df.corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0)
plt.title('Correlation Heatmap of Numerical Variables')
plt.tight_layout()
plt.savefig('outputs/fig5_correlation_heatmap.png', dpi=150)
plt.show()

# ---

# 6. Pairplot of selected numerical variables
selected_cols = ['Survived', 'Pclass', 'Age', 'Fare']
sns.pairplot(df[selected_cols], hue='Survived', palette='husl', diag_kind='hist')
plt.savefig('outputs/fig6_pairplot.png', dpi=150)
plt.show()

# ---

# 1. Descriptive statistics
df.describe(include='all').T

# ---

# 2. Frequency distribution (categorical variables)
for col in ['Survived', 'Pclass', 'Sex', 'Embarked']:
    print(f"\nFrequency distribution for {col}:")
    print(df[col].value_counts())
    print(df[col].value_counts(normalize=True).round(3) * 100, "%")

# ---

# 3. Correlation analysis
corr_matrix = df.select_dtypes(include=[np.number]).corr()
corr_matrix

# ---

# 4 & 5. Strongest positive and negative correlations (excluding self-correlation of 1.0)
corr_unstacked = corr_matrix.unstack()
corr_unstacked = corr_unstacked[corr_unstacked != 1.0]
strongest_positive = corr_unstacked.idxmax(), corr_unstacked.max()
strongest_negative = corr_unstacked.idxmin(), corr_unstacked.min()

```

```

print(f"Strongest positive correlation: {strongest_positive[0]} =
{strongest_positive[1]:.3f}")
print(f"Strongest negative correlation: {strongest_negative[0]} =
{strongest_negative[1]:.3f}")

# ---

# 1. Select suitable predictor variables
# We encode categorical variables and select features with plausible predictive value
ml_df = df.copy()

le_sex = LabelEncoder()
ml_df['Sex'] = le_sex.fit_transform(ml_df['Sex']) # male=1, female=0 (alphabetical)

le_embarked = LabelEncoder()
ml_df['Embarked'] = le_embarked.fit_transform(ml_df['Embarked'])

features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked', 'Has_Cabin']
X = ml_df[features]
y = ml_df['Survived']

print("Selected predictor variables:", features)
print("Target variable: Survived")

# ---

# 2. Split the dataset into training and testing sets (80/20 split)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print(f"Training set size: {X_train.shape[0]} passengers")
print(f"Testing set size: {X_test.shape[0]} passengers")

# ---

# 3. Train a Logistic Regression classifier
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
print("Model trained successfully.")

# ---

# 4. Predict on the testing data
y_pred = model.predict(X_test)

# ---

# 5. Compute Accuracy, Confusion Matrix, and Classification Report
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")

cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Did Not Survive', 'Survived'],
            yticklabels=['Did Not Survive', 'Survived'])
plt.title('Confusion Matrix - Logistic Regression')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.tight_layout()

```

```
plt.savefig('outputs/fig7_confusion_matrix.png', dpi=150)
plt.show()

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Did Not Survive',
'Survived']))

# ---

# Feature importance (coefficients) for interpretability
coef_df = pd.DataFrame({
    'Feature': features,
    'Coefficient': model.coef_[0]
}).sort_values('Coefficient', key=abs, ascending=False)
coef_df
```